



TRUSTSEC

Smart Contract Audit

Tenor – Morpho Midnight Contracts

11/07/2026

Executive summary

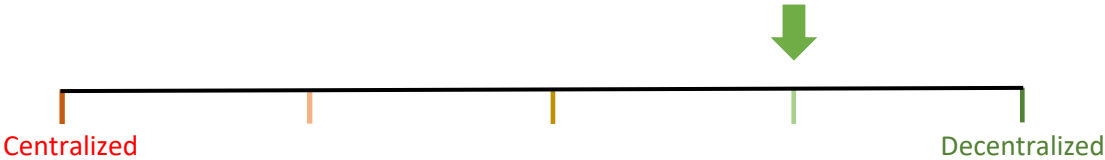


Category	Lending
Audited file count	34
Auditor	Trust HollaDieWaldfee
Time period	15/06/2026 - 07/07/2026

Findings

Severity	Total	Fixed	Acknowledged
High	0	0	0
Medium	7	4	3
Low	6	2	4

Centralization score



Signature



Digitally
signed by
Trust
Date:
2026.07.11
19:15:42
+03'00'

EXECUTIVE SUMMARY	1
DOCUMENT PROPERTIES	4
Versioning	4
Contact	4
INTRODUCTION	5
Scope	5
Repository details	6
About TrustSec	6
About the Auditors	6
Disclaimer	6
Methodology	7
QUALITATIVE ANALYSIS	8
FINDINGS	9
Medium severity findings	9
TRST-M-1 Zero-collateral Blue migrations should be supported but revert	9
TRST-M-2 <i>TenorRouter</i> take attempts lack per-action gas limits	10
TRST-M-3 Raw collateral LTV check does not track Midnight health for multi-collateral accounts	13
TRST-M-4 Collateral top-up ignores seller settlement fees on buy-offer taker path	14
TRST-M-5 <i>TenorRouter</i> can reset approvals required by in-flight Midnight callbacks	15
TRST-M-6 One-time sentinel sizing is vulnerable to callback-driven state changes, causing mis-sizing	17
TRST-M-7 <i>BaseMigrationRatifier</i> cannot represent economically viable exits for the keeper	19
Low severity findings	22
TRST-L-1 Midnight adapter max-withdraw sentinel is not clamped to pool withdrawable liquidity	22
TRST-L-2 Large non-sentinel maxFill skips fee adjuster pre-dispatch cap	23
TRST-L-3 LLTV=WAD vault liquidations can require pre-funding due to rounding	24
TRST-L-4 <i>TenorRouter</i> lacks continuous-fee slippage protection	26
TRST-L-5 Authorized delegate settings persist after Midnight authorizations are revoked	27
TRST-L-6 Dust donations can grief assets=type(uint256).max repayments	29
Additional recommendations	31
TRST-R-1 Use a consistent token approval pattern	31
TRST-R-2 Zero buyer-asset takes should be supported	31
TRST-R-3 Verify the <i>previewMint()</i> and <i>mint()</i> asset equality assumption	32
TRST-R-4 Code quality improvements	33
TRST-R-5 Documentation improvements	33

Centralization risks	36
TRST-CR-1 <i>MigrationIntentRatifier</i> owner	36
TRST-CR-2 Pausable interest rate policy owner	36
TRST-CR-3 Pausable interest rate policy pauser	37
Systemic risks	38
TRST-SR-1 External token risks and assumptions	38
TRST-SR-2 External protocol risks	38
TRST-SR-3 Borrower migration callbacks don't check account health	38

Document properties

Versioning

Version	Date	Description
0.1	07/07/2026	Client report
0.2	11/07/2026	Mitigation review

Contact

Trust

trust@trust-security.xyz

Introduction

TrustSec has conducted an audit at the customer's request. The audit is focused on uncovering security issues and additional bugs contained in the code defined in scope. Some additional recommendations have also been given when appropriate.

Scope

- `src/BaseMigrationRatifier.sol`
- `src/periphery/TenorRouter.sol`
- `src/periphery/libraries/TakeMathLib.sol`
- `src/bundler/MidnightAdapterBase.sol`
- `src/periphery/MidnightVaultExecutor.sol`
- `src/MigrationIntentRatifier.sol`
- `src/libraries/CallbackLib.sol`
- `src/callbacks/BorrowBlueToMidnightCallback.sol`
- `src/callbacks/MidnightSupplyCollateralCallback.sol`
- `src/bundler/TenorRouterAdapterBase.sol`
- `src/callbacks/BorrowMidnightToBlueCallback.sol`
- `src/policies/MarketMakingPolicy.sol`
- `src/policies/StaticRatePolicy.sol`
- `src/callbacks/BorrowMidnightRenewalCallback.sol`
- `src/IntentSettler.sol`
- `src/callbacks/MidnightSupplyVaultSharesCallback.sol`
- `src/callbacks/LendMidnightRenewalCallback.sol`
- `src/periphery/libraries/RouterLib.sol`
- `src/callbacks/LendMidnightToVaultCallback.sol`
- `src/libraries/CollateralTransferLib.sol`
- `src/callbacks/LendVaultToMidnightCallback.sol`
- `src/callbacks/MidnightWithdrawVaultSharesCallback.sol`
- `src/libraries/PriceLib.sol`
- `src/bundler/AuthorizationAdapter.sol`
- `src/policies/PausableStaticRatePolicy.sol`
- `src/libraries/LinearInterpolationLib.sol`
- `src/policies/PausableMarketMakingPolicy.sol`
- `src/policies/PausableBase.sol`
- `src/bundler/MigrationIntentRatifierAdapterBase.sol`
- `src/libraries/TenorMarketIdLib.sol`
- `src/bundler/TenorAdapter.sol`
- `src/libraries/RatePointLib.sol`
- `src/policies/FourWeekCadence.sol`
- `src/libraries/Constants.sol`

Note that functions within the in-scope contracts which are only used by out-of-scope contracts, such as *TakeMathLib.maxUnitsForSellerBudget()*, have not been reviewed. Their business logic can't be evaluated given that it entirely depends on the out-of-scope files.

Repository details

- **Repository URL:** <https://github.com/Shippoor-Labs/tenor-morpho-v2-contracts-2>
- **Commit hash:** ec9e656607de66684d5e500be4f8bc8574a24a8a

Since the initial audit, the codebase was published in the **tenor-contracts** repository.

- **Mitigation repository URL:** <https://github.com/tenor-labs/tenor-contracts>
- **Mitigation hash:** d3756dd834e28c9c57d7191f39482ad218c48df0

About TrustSec

TrustSec has been established by top-end blockchain security researcher Trust, in order to provide high quality auditing services. Trust is the leading auditor at competitive auditing service Code4rena, reported several critical issues to Immunefi bug bounty platform and is currently a Code4rena judge.

About the Auditors

Trust has established a dominating presence in the smart contract security ecosystem since 2022. He is a resident on the Immunefi, Sherlock and C4 leaderboards and is now focused in auditing and managing audit teams under TrustSec. When taking time off auditing & bug hunting, he enjoys assessing bounty contests in C4 as a Supreme Court judge.

HollaDieWaldfee is a distinguished security expert with a track record of multiple first places in competitive audits. He is a Lead Auditor at TrustSec and Senior Watson at Sherlock.

Disclaimer

Smart contracts are an experimental technology with many known and unknown risks. TrustSec assumes no responsibility for any misbehavior, bugs or exploits affecting the audited code or any part of the deployment phase.

Furthermore, it is known to all parties that changes to the audited code, including fixes of issues highlighted in this report, may introduce new issues and require further auditing.

Methodology

In general, the primary methodology used is manual auditing. The entire in-scope code has been deeply looked at and considered from different adversarial perspectives. Any additional dependencies on external code have also been reviewed.

Qualitative analysis

Metric	Rating	Comments
Code complexity	Good	Project generally kept code simple, reducing attack surface.
Documentation	Excellent	Project is well documented.
Best practices	Good	Project generally adheres to best practices.
Centralization risks	Excellent	Protocol is permissionless, only using privileged roles for fee settings (which must be accepted by the user-supplied rate bounds), and opt-in pausable interest rate policies.

Findings

Medium severity findings

TRST-M-1 Zero-collateral Blue migrations should be supported but revert

- **Category:** Logical issues
- **Source:** BorrowBlueToMidnightCallback.sol, BorrowMidnightToBlueCallback.sol
- **Status:** Fixed

Description

BorrowBlueToMidnightCallback documents that small partial fills can migrate debt while the pro-rata collateral transfer rounds to zero.

```
/// @dev On small partial fills, the pro-rata collateral transfer can round to zero
even though debt is
/// migrated, temporarily increasing the target position's LTV until the position is
fully migrated.
```

The implementation computes the migrated collateral pro-rata for non-final fills, then always calls Morpho Blue to withdraw collateral and Midnight to supply it.

```
uint256 collateralMigrated = isFinalFill ? v1Collateral :
v1Collateral.mulDivDown(repayBudget, v1Debt);

MORPHO_BLUE.withdrawCollateral(sourceMarketParams, collateralMigrated, seller,
address(this));
IERC20(sourceMarketParams.collateralToken).forceApprove(address(MORPHO_MIDNIGHT),
collateralMigrated);
MORPHO_MIDNIGHT.supplyCollateral(market, collateralIndex, collateralMigrated,
seller);
```

When **collateralMigrated == 0**, the callback still calls *withdrawCollateral()* with zero assets. Morpho Blue rejects zero-asset collateral withdrawals.

```
function withdrawCollateral(MarketParams memory marketParams, uint256 assets, address
onBehalf, address receiver)
    external
{
    Id id = marketParams.id();
    require(market[id].lastUpdate != 0, ErrorsLib.MARKET_NOT_CREATED);
    require(assets != 0, ErrorsLib.ZERO_ASSETS);
    ...
}
```

As a result, the documented tiny-partial-fill behavior is not reachable. The fill fails atomically instead of migrating debt with zero collateral.

The same issue exists in the opposite direction. *BorrowMidnightToBlueCallback* also documents that small partial fills can withdraw zero collateral while migrating debt.

```

/// @dev On small partial fills, the pro-rata collateral withdrawal can round to zero
even though debt is migrated,
/// temporarily increasing the target Blue position's LTV until the position is fully
migrated.

```

The callback computes the collateral transfer amount and always calls *withdrawCollateral()* on Midnight followed by *supplyCollateral()* on Morpho Blue.

```

uint256 collateralTransferred =
    sourceDebtAfter == 0 ? sourceCollateral : sourceCollateral.mulDivDown(units,
sourceDebtBefore);

MORPHO_MIDNIGHT.withdrawCollateral(market, collateralIndex, collateralTransferred,
buyer, address(this));

IERC20(callbackData.targetMarketParams.collateralToken)
    .forceApprove(address(MORPHO_BLUE), collateralTransferred);
MORPHO_BLUE.supplyCollateral(callbackData.targetMarketParams, collateralTransferred,
buyer, "");

```

Midnight permits *withdrawCollateral()* with zero assets, but Morpho Blue rejects the following zero-asset *supplyCollateral()* call.

```

function supplyCollateral(MarketParams memory marketParams, uint256 assets, address
onBehalf, bytes calldata data)
    external
{
    Id id = marketParams.id();
    require(market[id].lastUpdate != 0, ErrorsLib.MARKET_NOT_CREATED);
    require(assets != 0, ErrorsLib.ZERO_ASSETS);
    ...
}

```

This also blocks a non-dust route where the source Midnight account uses one collateral while the target Blue market uses a different collateral that is listed in the Midnight market but not currently supplied by the borrower. If the borrower has manually pre-seeded enough collateral in Blue, the Blue borrow could be healthy without transferring any collateral from Midnight. The callback still attempts zero-asset *supplyCollateral()* before borrowing, so the route reverts before Blue can enforce its own health check.

Recommended mitigation

Skip zero-amount collateral operations and let the destination protocol's existing health checks decide whether the debt-only partial migration is valid.

Team response

Fixed in [PR11](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-2 *TenorRouter* take attempts lack per-action gas limits

- **Category:** Griefing issues
- **Source:** TenorRouter.sol

- **Status:** Acknowledged

Description

TenorRouter is designed to execute a batch of candidate actions and continue past individually failing actions when **Action.allowRevert** is enabled. In the main loop, a failed dispatch emits **ActionReverted** and the router proceeds to the next action.

```
if (!success) {
  if (!action.allowRevert) revert ActionFailed(i, reason);
  emit ActionReverted(i, reason);
  continue;
}
```

The dispatch paths implement this behavior with **try/catch** around either *IntentSettler.take()* or *Midnight.take()*.

```
try _INTENT_SETTLER.take(
  offerData,
  takeUnits,
  initiator,
  d.receiver,
  d.takerCallback,
  d.takerCallbackData,
  d.intent,
  d.intentRatifierData
) returns (uint256 r0, uint256 r1) {
  return (true, r0, r1, takeUnits, marketId, "");
} catch (bytes memory reason) {
  return (false, 0, 0, 0, marketId, reason);
}

try _MORPHO_MIDNIGHT.take(
  action.offer,
  d.ratifierData,
  takeUnits,
  initiator,
  d.receiverIfTakerIsSeller,
  d.takerCallback,
  d.takerCallbackData
) returns (uint256 r0, uint256 r1) {
  if (routerIsPayer) SafeERC20.forceApprove(IERC20(loanToken),
address(_MORPHO_MIDNIGHT), 0);
  return (true, r0, r1, takeUnits, marketId, "");
} catch (bytes memory reason) {
  if (routerIsPayer) SafeERC20.forceApprove(IERC20(loanToken),
address(_MORPHO_MIDNIGHT), 0);
  return (false, 0, 0, 0, marketId, reason);
}
```

Neither external call specifies a gas limit. As a result, a failing action can consume nearly all gas forwarded to the take attempt before reverting. [EIP-150](#) leaves the router with only 1/64th of the pre-call gas at the external call boundary, so the catch block may still execute but later valid actions may no longer have enough gas to run.

This undermines the purpose of **allowRevert**. A route can be constructed with one or more fallback offers, but a malicious or faulty early offer can still consume the gas intended for those later offers. The batch then runs out of gas before reaching the well-functioning actions even though they were intended to provide fallback liquidity.

A malicious maker can exploit this by posting an attractive offer whose ratifier or callback burns gas and then reverts. Since Midnight offers support arbitrary take-time callbacks,

callback behavior cannot be reliably bounded by quote-time simulation or a static offchain allowlist. The router already documents that maker-supplied components may be gas-expensive.

```
/// @dev Maker-supplied policies, resolvers and clamps are untrusted code; keepers
must treat quoting and dispatch as
/// potentially reverting or gas-expensive.
```

That warning is useful for operators, but it does not enforce an execution-time gas reserve for subsequent actions. A transaction-level gas limit only bounds the entire batch; it does not isolate one failed action from the gas budget needed by later fallback actions.

Morpho Midnight deliberately does not restrict the gas that is passed along to *onBuy()* and *onSell()* callbacks, making gas consumption validation the responsibility of the periphery contracts. Furthermore, the Tenor codebase includes callbacks with variable gas costs, e.g., **vault** gas consumption in *LendVaultToMidnightCallback*. These callbacks are meant to be used for *TenorRouter* takes. As such, Tenor must supply the infrastructure to protect against arbitrary gas consumption.

Recommended mitigation

Add per-action gas controls to router dispatch. For example, extend **Action** with a **gasLimit** field, or add a global **maxGasPerAction** execution parameter, then call the external take attempts with an explicit gas budget.

```
try _MORPHO_MIDNIGHT.take{gas: gasLimit}{
    action.offer,
    d.ratifierData,
    takeUnits,
    initiator,
    d.receiverIfTakerIsSeller,
    d.takerCallback,
    d.takerCallbackData
} returns (uint256 r0, uint256 r1) {
    ...
} catch (bytes memory reason) {
    ...
}
```

The router should also verify before calling *take()* that enough gas is available. If the requested gas limit is higher than the gas that can be forwarded, the EVM silently clamps the forwarded gas rather than reverting. Without an explicit availability and reserve check, an action may receive less gas than intended.

If the router cannot safely provide the configured gas limit, that action should fail and be handled by the existing **allowRevert** logic while preserving gas for later actions.

Team response

Acknowledged and [documented](#).

TRST-M-3 Raw collateral LTV check does not track Midnight health for multi-collateral accounts

- **Category:** Logical issues
- **Source:** MidnightSupplyCollateralCallback.sol
- **Status:** Fixed

Description

MidnightSupplyCollateralCallback lets the seller configure **maxLtv** as a post-supply sanity check. The callback supplies the configured collateral amounts and then computes account LTV as debt over total raw collateral value.

```
uint256 collateralValue;
uint256 collateralsLength = market.collateralParams.length;

for (uint256 i = 0; i < collateralsLength;) {
    uint256 collateralAmount = MORPHO_MIDNIGHT.collateral(marketId, borrower, i);
    if (collateralAmount > 0) {
        uint256 price = IOracle(market.collateralParams[i].oracle).price();
        collateralValue += collateralAmount.mulDivDown(price, ORACLE_PRICE_SCALE);
    }
    unchecked {
        ++i;
    }
}

return debt.mulDivUp(WAD, collateralValue);
```

This check is LLTV-agnostic. It treats all collateral value as contributing equally to account safety, while Midnight's actual health check discounts each collateral by its configured LLTV.

```
maxDebt += _position.collateral[i].mulDivDown(price, ORACLE_PRICE_SCALE)
    .mulDivDown(collateralParam.lltv, WAD);

return maxDebt >= debt;
```

For a single-collateral account, or for an account whose collateral value weights are fixed, raw LTV can be converted to Midnight capacity utilization by a constant factor.

```
effectiveLLTV = sum(collateralValue_i * lltv_i) / sum(collateralValue_i)
capacityUtilization = rawLTV / effectiveLLTV
```

In multi-collateral markets, that conversion factor is not stable. It changes with the account's collateral composition and with relative oracle prices. As a result, a fixed **maxLtv** threshold does not generally express a fixed distance from Midnight's liquidation threshold.

For example, consider a market with two collaterals where collateral A has value 100 and LLTV 90%, collateral B has value 100 and LLTV 50%, and debt is 100. The callback reports raw LTV of 50%, but Midnight borrow capacity is 140 and capacity utilization is 71.4%.

```
rawLTV = 100 / (100 + 100) = 50%
borrowCapacity = 100 * 90% + 100 * 50% = 140
capacityUtilization = 100 / 140 = 71.4%
```

If collateral B's price doubles, the same raw LTV threshold maps to a different liquidation buffer.

```
borrowCapacity = 100 * 90% + 200 * 50% = 190  
capacityUtilization at 50% raw LTV = 150 / 190 = 78.9%
```

The same **maxLtv = 50%** threshold that corresponded to 71.4% capacity utilization under the original price mix now corresponds to 78.9% capacity utilization. The account can therefore be materially closer to Midnight's liquidation threshold while still satisfying the same raw LTV limit.

This does not bypass Midnight's final solvency check. *Midnight.take()* still requires the seller to be healthy after the callback. The issue is that the callback's user-facing risk control is a raw LTV check, not an account-health or liquidation-buffer check, so it can provide weaker protection than expected in multi-collateral markets.

Recommended mitigation

Replace the raw LTV check with a check against Midnight's LLTV-weighted borrow capacity. Compute the same **maxDebt** value as *Midnight.isHealthy()* and enforce a configured maximum capacity utilization.

Team response

Fixed in [PR615](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-4 Collateral top-up ignores seller settlement fees on buy-offer taker path

- **Category:** Logical issues
- **Source:** MidnightSupplyVaultSharesCallback.sol
- **Status:** Fixed

Description

MidnightSupplyVaultSharesCallback documents **additionalDepositPercent** as an additional deposit equal to an extra share of the seller's received **sellerAssets**.

```
/// @param additionalDepositPercent The extra share of sellerAssets the seller must  
provide, in WAD (e.g. 0.1e18 =  
/// 10% extra).
```

However, the implementation calculates the extra pull from the filled **units** and **callbackData.tick**.

```
uint256 rate =  
TickLib.tickToPrice(callbackData.tick).mulDivUp(callbackData.additionalDepositPercent  
, WAD);  
amountFromSeller = units.mulDivUp(rate, WAD);
```

For the usual maker-side SELL-offer path, Midnight computes seller proceeds from the raw offer price, so this uses the same economic base as **sellerAssets** apart from rounding. In the

BUY-offer taker path, the callback is supplied as **takerCallback**, and the user is the taker. In that path Midnight deducts settlement fees before passing **sellerAssets** to *onSell()*.

```
uint256 sellerPrice = offer.buy ? offerPrice - _settlementFee : offerPrice;
uint256 sellerAssets = offer.buy ? units.mulDivDown(sellerPrice, WAD) :
units.mulDivUp(sellerPrice, WAD);
```

The callback still computes the top-up from **offerPrice**, not from the settlement-fee-adjusted **sellerAssets**. As a result, takers on BUY offers are charged more than the documented percentage of their actual proceeds. The same mismatch can also revert valid fills when the seller approved only the documented amount.

Recommended mitigation

Calculate the additional deposit from the actual **sellerAssets** value supplied by Midnight.

```
amountFromSeller = sellerAssets.mulDivUp(callbackData.additionalDepositPercent, WAD);
```

This matches the documented meaning, accounts for settlement fees, and removes **callbackData.tick** from the top-up calculation.

Team response

Fixed in [PR614](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-5 *TenorRouter* can reset approvals required by in-flight Midnight callbacks

- **Category:** Reentrancy issues
- **Source:** MidnightAdapterBase.sol, TenorRouter.sol
- **Status:** Fixed

Description

TenorRouter._dispatchMidnightTake() temporarily approves Midnight when the router itself is the payer for a direct **MIDNIGHT_TAKE**, then resets that approval to zero after the take returns or reverts.

```
bool routerIsPayer = d.takerCallback == address(0) && !action.offer.buy;
if (routerIsPayer) {
    SafeERC20.forceApprove(IERC20(loanToken), address(_MORPHO_MIDNIGHT),
type(uint256).max);
}

try _MORPHO_MIDNIGHT.take(...) returns (uint256 r0, uint256 r1) {
    if (routerIsPayer) SafeERC20.forceApprove(IERC20(loanToken),
address(_MORPHO_MIDNIGHT), 0);
    return (true, r0, r1, takeUnits, marketId, "");
} catch (bytes memory reason) {
    if (routerIsPayer) SafeERC20.forceApprove(IERC20(loanToken),
address(_MORPHO_MIDNIGHT), 0);
    return (false, 0, 0, 0, marketId, reason);
}
```

In the deployed adapter composition, *TenorAdapter* inherits both *MidnightAdapterBase* and *TenorRouterAdapterBase*. Therefore, the **loanToken -> Midnight** allowance used by router actions is the same allowance used by Midnight adapter operations.

This is unsafe when *execute()* is reached in a reentrant context from a Midnight callback whose outer operation still depends on that allowance.

For example, *midnightRepay()* approves Midnight and calls *Midnight.repay()*.

```
SafeERC20.forceApprove(IERC20(market.loanToken), address(MORPHO_MIDNIGHT),
type(uint256).max);
address callback = data.length > 0 ? address(this) : address(0);
MORPHO_MIDNIGHT.repay(market, units, onBehalf, callback, data);
```

Midnight.repay() invokes the callback before pulling repayment tokens from the payer.

```
if (callback != address(0)) {
    require(
        IRepayCallback(callback).onRepay(id, market, units, onBehalf, data) ==
        CALLBACK_SUCCESS,
        WrongRepayCallbackReturnValue()
    );
}
SafeTransferLib.safeTransferFrom(market.loanToken, payer, address(this), units);
```

If the repay callback reenters *Bundler3* and executes a same-token direct **MIDNIGHT_TAKE** through the **routerIsPayer** branch, the router resets the adapter's Midnight allowance to zero before control returns to *Midnight.repay()*. The final repayment pull then reverts even though *midnightRepay()* approved Midnight at the start of the flow.

The same call-ordering issue exists for *midnightFlashLoan()*. The adapter approves Midnight before calling *Midnight.flashLoan()*.

```
if (IERC20(tokens[i]).allowance(address(this), address(MORPHO_MIDNIGHT)) !=
type(uint256).max) {
    SafeERC20.forceApprove(IERC20(tokens[i]), address(MORPHO_MIDNIGHT),
type(uint256).max);
}
MORPHO_MIDNIGHT.flashLoan(tokens, assets, address(this), data);
```

Midnight.flashLoan() calls *onFlashLoan()* and only then pulls the flash-loaned assets back.

```
IFlashLoanCallback(callback).onFlashLoan(msg.sender, tokens, assets, data);
for (uint256 i = 0; i < tokens.length; i++) {
    SafeTransferLib.safeTransferFrom(tokens[i], callback, address(this), assets[i]);
}
```

A same-token nested **MIDNIGHT_TAKE** from *onFlashLoan()* can therefore clear the allowance required for the final flash-loan repayment pull as well.

As a result, valid *TenorRouter* bundles can become unexecutable. The repay callback failure path can be summarized as:

```

midnightRepay()
-> adapter approves Midnight for max loanToken
-> Midnight.repay() calls adapter.onRepay() before pulling repayment tokens
-> adapter.onRepay() reenters Bundler3 with the callback bundle
-> adapter.execute() runs a same-token MIDNIGHT_TAKE
-> TenorRouter._dispatchMidnightTake() enters the routerIsPayer branch
-> the nested take returns and TenorRouter resets the adapter's Midnight allowance to 0
-> control returns to Midnight.repay()
-> Midnight tries to pull REPAY_UNITS from the adapter
-> transferFrom() reverts because the allowance is 0

```

Recommended mitigation

Do not reset the router's Midnight allowance to zero after a **routerIsPayer** take. Leaving the max approval in place matches the existing adapter pattern for Midnight repayment and flash-loan flows. For standard ERC-20 implementations, a **type(uint256).max** approval is not practically consumed by inner operations.

Add an implementation note for tokens with capped or nonstandard allowance behavior, such as UNI- or COMP-style tokens where a **uint256.max** approval clamps to **uint96.max**. For those tokens, inner same-token operations can consume a finite allowance in practice, so integrations should either avoid relying on shared in-flight approvals or use a token-compatible maximum with sufficient headroom.

Team response

Fixed in [PR8](#).

Mitigation review

Verified, the approval reset has been removed.

TRST-M-6 One-time sentinel sizing is vulnerable to callback-driven state changes, causing mis-sizing

- **Category:** Logical issues
- **Source:** TenorRouterAdapterBase.sol, TenorRouter.sol
- **Status:** Acknowledged

Description

TenorRouterAdapterBase documents **FILL_UNITS** sentinel sizing as a side-aware cap that matches the initiator's existing debt or credit and prevents position overshoot.

```

/// @dev FILL_UNITS -> initiator's debt (buyer-side) or credit (seller-side); side-
aware so
/// the resolved cap matches the existing position the action would close,
preventing overshoot.
function _resolveSentinel(uint8 fillIndex, Action[] calldata actions) internal
returns (uint256) { ... }

```

However, *_executeResolvingSentinels()* resolves **type(uint256).max** sentinels once before the batch starts. The resolved **maxFill** and **minFill** values are then used as fixed limits for all later actions.

```

if (actions.length > 0 && (maxFill == type(uint256).max || minFill ==
type(uint256).max)) {
    address initiator = _initiator();
    if (!_isFirstParty(actions[0], initiator)) revert SentinelRequiresFirstParty();
    uint8 fillIndex = uint8(params.fillAxis);
    uint256 resolved = _resolveSentinel(fillIndex, actions);
    if (maxFill == type(uint256).max) maxFill = resolved;
    if (minFill == type(uint256).max) minFill = resolved;
}

```

This is unsafe because each successful *Midnight.take()* can execute untrusted callback code before the next routed action. That callback can change the initiator's own position through a nested take, or it can change shared market state through liquidations. The router continues to size later actions from the initial sentinel snapshot.

For buyer-side debt close-outs, Midnight nets bought units against debt first and then credits any excess. If the initial sentinel cap is stale, a later routed action can cross the account into credit while still staying within the router's top-level **maxFill**.

In the following scenario, a malicious callback causes the victim's position to overshoot into credit:

1. The borrower starts with **100e18** debt and submits a router batch with **FillAxis.UNITS** and **maxFill = type(uint256).max**. The adapter resolves **maxFill** to **100e18** once at the beginning of the batch.
2. Routed action T1 buys **50e18** units for the user, reducing the debt to **50e18**.
3. T1's callback executes a nested *Midnight.take()* T2 that buys another **40e18** units for the same user. The actual debt is now **10e18**, but the router has counted only the **50e18** from T1. T2 could for example be a migration the user has authorized in *BaseMigrationRatifier*.
4. Routed action T3 is still allowed to buy **50e18** units because the fixed router budget has **50e18** remaining. T3 repays the remaining **10e18** debt and creates **40e18** credit. Router totals equal the original **100e18** cap, so **FillOvershoot** does not trigger.

Setting router-level **reduceOnly** changes the impact but does not remove the root cause. The same stale sentinel can instead revert the whole batch.

The opposite overshoot from credit to debt can also be achieved when a callback realizes bad debt in the same market and reduces the user's updated credit through the **lossFactor**.

```

if (badDebt > 0) {
    _position.debt -= uint128(badDebt);
    uint256 _totalUnits = _marketState.totalUnits;
    uint256 _lossFactor = _marketState.lossFactor;
    _marketState.lossFactor = UtilsLib.toUint128(
        type(uint128).max - (type(uint128).max - _lossFactor).mulDivUp(_totalUnits -
badDebt, _totalUnits)
    );
    _marketState.totalUnits = UtilsLib.toUint128(_totalUnits - badDebt);
}

uint256 postSlashCredit = _lastLossFactor < type(uint128).max
    ? credit.mulDivDown(type(uint128).max - marketState[id].lossFactor,
type(uint128).max - _lastLossFactor)
    : 0;

```

In that case the initial **maxFill** or **minFill** can become too large for the current credit position. If **reduceOnly** is false, the batch can cross into the opposite-side debt position. If **reduceOnly** is true, the router reverts the entire batch. This shows that **reduceOnly** is not a mitigation to safely use the sentinel semantics. And as the comment quoted above indicates, **minFill** and **maxFill** sentinels are meant to be sufficient on their own to prevent overshoot.

Recommended mitigation

The root cause is that the user's intention is mapped to **minFill** and **maxFill** only once at the start of the batch and is not re-evaluated after each action. There are two units-sizing intentions to support: closing out all debt and closing out all credit. If the re-sizing after each action is implemented, actions that cross into the opposite position should be reverted, continuing the batch with the next action which may not be malicious.

An alternative solution is to make this adaptive position sizing the responsibility of the *ITakeClamp* interface which runs before every take. However, this approach does not allow rolling back an action once it has already crossed into the opposite side and would therefore have to revert the whole batch.

Finally, sentinel semantics may be weakened, making it clear that sentinels only evaluate initial debt and credit but can't provide any guarantee on their own about the resulting exposure.

Team response

Acknowledged and [documented](#).

TRST-M-7 *BaseMigrationRatifier* cannot represent economically viable exits for the keeper

- **Category:** Economic issues
- **Source:** BaseMigrationRatifier.sol
- **Status:** Acknowledged

Description

BaseMigrationRatifier uses the remaining source-market time as the rate-check duration for Midnight exit flows. That duration is zero once the source Midnight market has reached maturity.

```

function _computeDuration(address callback, uint256 sourceMaturity, uint256
targetMaturity)
    internal
    view
    returns (uint256)
{
    if (callback == BORROW_MIDNIGHT_RENEWAL_CALLBACK || callback ==
LEND_MIDNIGHT_RENEWAL_CALLBACK) {
        uint256 effectiveStart = block.timestamp > sourceMaturity ? block.timestamp :
sourceMaturity;
        return targetMaturity - effectiveStart;
    } else if (callback == BORROW_BLUE_TO_MIDNIGHT_CALLBACK || callback ==
LEND_VAULT_TO_MIDNIGHT_CALLBACK) {
        return targetMaturity - block.timestamp;
    } else {
        return UtilsLib.zeroFloorSub(sourceMaturity, block.timestamp);
    }
}

```

PriceLib.computePrice() returns par when the duration is zero. The buyer-side limit becomes an effective price ceiling of **WAD**, and the seller-side limit becomes an effective price floor of **WAD**.

```

uint256 denominator = WAD + ratePerSecond * durationSeconds;
return isBuy ? WAD.mulDivDown(WAD, denominator) : WAD.mulDivUp(WAD, denominator);
if (isBuy) {
    return assets * WAD <= units * price;
} else {
    return assets * WAD >= units * price;
}

```

The ratifier also folds Midnight settlement fees into the effective execution price. This makes post-maturity keeper exits impossible to represent economically. In a *BorrowMidnightToBlueCallback* exit, the user is the buyer. A keeper selling credit at par as the maker would quote **offerPrice = WAD**, so the buyer pays **WAD + settlementFee**. The ratifier rejects this because the post-maturity buyer-side limit is still **WAD**. To pass the ratifier, the keeper would need to sell below par by at least the settlement fee, which is not an economically rational last-resort fill. The same issue occurs when the user is the maker. In that case an economically viable offer has an **offerPrice** above **WAD** such that **offerPrice - settlementFee >= WAD** which is rejected by Midnight.

The same par-price collapse affects lender exits. In a *LendMidnightToVaultCallback* exit, the user is the seller. At maturity the ratifier requires the user to receive at least **WAD** per unit, but a keeper buy offer pays the seller **offerPrice - settlementFee**. Making the seller whole would require the keeper buyer to quote above par by the settlement fee, while a par quote makes the user receive less than the ratified price. Similarly, when the keeper takes a user sell offer, an economically viable offer would need to have a price below **WAD** which the ratifier rejects.

This flow where matured Midnight positions are migrated into Blue or Vaults is intended as a last resort to prevent liquidations when a migration before maturity was not possible. But due to the pricing constraints, economically viable migrations are rejected by the ratifier.

Finally, even before maturity, buyer-side exits can require an above-par effective buyer price when the remaining fair-market discount to maturity is smaller than the settlement fee. In that case, making the keeper seller whole requires the buyer to pay **fairPrice + settlementFee**

> **WAD**, but the ratifier's interest-rate model can only produce prices less than or equal to **WAD**.

Recommended mitigation

The limitation is structural since pricing below par for sells and above par for buys at or after maturity is incompatible with the interest rate-based model. The ratifier only lets users specify an interest rate curve. There is at least one additional information necessary which is the accepted offset from par (**par + offset** for buys, and **par – offset** for sells) at different points in the curve.

Furthermore, in the flow where the keeper is the seller and the offer is a buy offer by the user, the economically viable price would require an **offerPrice > WAD** such that **offerPrice – settlementFee >= WAD**. But this is not allowed by Midnight. This flow is only compatible with the keeper as the maker, unless the keeper is compensated via a fee transfer outside of Midnight.

Team response

Acknowledged and [documented](#).

Low severity findings

TRST-L-1 Midnight adapter max-withdraw sentinel is not clamped to pool withdrawable liquidity

- **Category:** Race conditions
- **Source:** MidnightAdapterBase.sol
- **Status:** Acknowledged

Description

MidnightAdapterBase.midnightWithdraw() is a direct *Bundler3* adapter primitive for withdrawing Midnight credit on behalf of the bundle initiator. When **units == type(uint256).max**, the adapter resolves the sentinel to the initiator's full updated credit.

```
function midnightWithdraw(Market calldata market, uint256 units, address receiver)
external onlyBundler3 {
    address onBehalf = initiator();
    if (units == type(uint256).max) {
        bytes32 id = IdLib.toId(market, INITIAL_CHAIN_ID, address(MORPHO_MIDNIGHT));
        (units,,) = MORPHO_MIDNIGHT.updatePositionView(market, id, onBehalf);
    }

    require(units != 0, ErrorsLib.ZeroAmount());

    MORPHO_MIDNIGHT.withdraw(market, units, onBehalf, receiver);
}
```

The resolved amount is not capped by the market's pool-level **withdrawable** liquidity. Midnight then subtracts both the user's credit and the market's withdrawable amount.

```
_position.credit -= UutilsLib.toUint128(units);
_marketState.withdrawable -= UutilsLib.toUint128(units);
_marketState.totalUnits -= UutilsLib.toUint128(units);
```

If the user's updated credit is greater than the currently withdrawable pool liquidity, *Midnight.withdraw()* reverts by underflowing **_marketState.withdrawable**.

This can happen naturally in shared-liquidity markets because **withdrawable** is global market liquidity, not lender-specific liquidity. Another lender can withdraw first and reduce the shared **withdrawable** amount before the victim's bundled *midnightWithdraw()* executes.

The router clamp system handles this in some take-based flows. For example, *LendMidnightRenewalClamp* explicitly caps the source withdrawal budget by **min(sourceCredit, poolWithdrawable)**.

```
uint256 poolWithdrawable = MORPHO_MIDNIGHT.withdrawable(data.sourceMarketId);
uint256 withdrawableAssets = UutilsLib.min(sourceCredit, poolWithdrawable);
```

However, *MidnightAdapterBase.midnightWithdraw()* is not a *TenorRouter* action and is not protected by an *ITakeClamp*. It remains a lower-level *Bundler3* primitive with no "withdraw max currently available" mode.

Recommended mitigation

Change *midnightWithdraw()* to support an explicit **capToWithdrawable** flag that caps the resolved withdrawal amount to the market's current **withdrawable** liquidity. Callers that want the existing full-withdrawal behavior can pass **capToWithdrawable = false**. Callers that want "withdraw as much as currently available" can pass **capToWithdrawable = true**, preventing shared-liquidity races from reverting the whole bundle when partial withdrawal is acceptable.

Team response

Acknowledged and [documented](#).

TRST-L-2 Large non-sentinel maxFill skips fee adjuster pre-dispatch cap

- **Category:** Logical issues
- **Source:** TenorRouter.sol
- **Status:** Fixed

Description

ICallbackFeeAdjuster.beforeDispatch() is documented as the router-side pre-dispatch sizer that caps **takeUnits** so the effective fill stays within the remaining budget.

```
/// @notice Pre-dispatch sizer: caps `takeUnits` so the effective `fillIndex` fill
/// stays within `remainingBudget`.
/// @dev The cap is conservative: it may underfill by a rounding tolerance but never
/// overshoots.
function beforeDispatch(Offer calldata offer, uint8 fillIndex, uint256
remainingBudget, bytes calldata data)
    external
    view
    returns (uint256 takeUnits);
```

However, *TenorRouter._capTakeUnits()* skips the entire budget-to-units conversion block when **remaining** is greater than or equal to **type(uint256).max / WAD**.

```
if (remaining < type(uint256).max / WAD) {
    uint256 cap = action.feeAdjuster != address(0)
        ? ICallbackFeeAdjuster(action.feeAdjuster)
          .beforeDispatch(action.offer, fillIndex, remaining,
action.feeAdjusterData)
        : RouterLib.budgetToUnits(_MORPHO_MIDNIGHT, marketId, action.offer,
fillIndex, remaining);
    takeUnits = UtilsLib.min(takeUnits, cap);
}
```

The action's **takeUnits** is still capped by offer capacity and the optional **clamp**, but any cap that the fee adjuster would have returned is bypassed.

The adapter makes the exact value **type(uint256).max** special by resolving it as a sentinel.

```
if (actions.length > 0 && (maxFill == type(uint256).max || minFill ==
type(uint256).max)) {
    ...
    if (maxFill == type(uint256).max) maxFill = resolved;
    if (minFill == type(uint256).max) minFill = resolved;
}
```

But values below the sentinel are passed through unchanged. For example, **maxFill = type(uint256).max - 1** is not resolved, yet it still satisfies the router's skip condition.

For the shipped *CallbackFeeAdjuster*, the practical impact is limited. A mathematically correct fee-aware inversion of a remaining budget $\geq \text{type(uint256).max} / \text{WAD}$ should also return a units cap with roughly the same magnitude such that it is not a constraint in practice.

However, the interface is broader than the concrete implementation. A custom *ICallbackFeeAdjuster* can use *beforeDispatch()* to impose a stricter pre-dispatch cap. That cap is not enforced in the skipped case. The resulting batch can either fill more units than the adjuster intended or later fail closed through **FillOvershoot** if post-dispatch accounting exceeds **maxFill**.

Recommended mitigation

Do not skip the fee adjuster hook. If a fee adjuster is configured, always call *beforeDispatch()* and require it to handle large inputs safely. It is more important to always call the *CallbackFeeAdjuster* than to avoid math overflows. The latter is not a responsibility of the *TenorRouter* but of the *CallbackFeeAdjuster*.

Team response

Fixed in [PR25](#).

Mitigation review

Verified, the fee adjuster hook is now called unconditionally. To remediate the overflow, *TenorRouter* checks **maxFill** and reverts if it exceeds **type(uint128).max**. This is a sufficient limit for practical purposes.

TRST-L-3 LLTV=WAD vault liquidations can require pre-funding due to rounding

- **Category:** Rounding issues
- **Source:** MidnightVaultExecutor.sol
- **Status:** Acknowledged

Description

MidnightVaultExecutor.onLiquidate() is intended to self-fund a vault-share collateral liquidation by redeeming the seized vault shares and then approving Midnight to pull the repaid units.

```
IERC4626(vault).redeem(seizedShares, address(this), address(this));  
_fundRepay(market.loanToken, repaidUnits, liquidator);
```

_fundRepay() uses the executor's full loan-token balance, refunds any surplus, and approves Midnight for exactly **repaidUnits**.

```

uint256 balance = IERC20(loanToken).balanceOf(address(this));
if (balance > repaidUnits) {
    IERC20(loanToken).safeTransfer(surplusReceiver, balance - repaidUnits);
}
IERC20(loanToken).forceApprove(address(MORPHO_MIDNIGHT), repaidUnits);

```

This assumes the redeemed vault assets are at least **repaidUnits**. For vault-share collateral with **lltv == WAD**, that assumption can fail due to rounding.

When **lltv == WAD**, Midnight's **maxLif** is also **WAD**, so the liquidation incentive factor has no buffer. In the **repaidUnits** liquidation path, Midnight rounds seized collateral down.

```

seizedAssets = repaidUnits.mulDivDown(lif, WAD).mulDivDown(ORACLE_PRICE_SCALE,
liquidatedCollatPrice);

```

The executor then redeems exactly those rounded-down shares through ERC-4626. *redeem()* can also round down the returned assets. The executor can receive slightly less than **repaidUnits**, but Midnight still pulls the full repayment amount after the callback.

If the executor has no prefunded loan-token balance, the final transfer can revert even though the liquidation is otherwise valid.

For example, with **lltv = WAD**, **lif = WAD**, **repaidUnits = 500e18**, and an accurate vault-share price of **1.1e36**, the executor is one wei short.

```

seizedShares = floor(500e18 / 1.1) = 454.545...e18 rounded down
redeemedAssets = floor(seizedShares * 1.1) = 500e18 - 1 wei

```

This does not make every such position permanently unliquidatable. A liquidator can atomically transfer a small loan-token buffer to the executor before calling *liquidate()*, because *_fundRepay()* counts the executor's full loan-token balance. This prefunding path uses the loan token, not vault shares, so it is not blocked by the vault share gate.

However, the current executor interface and docs do not make this prefunding requirement explicit. The executor is documented as pass-through custody, and balances left across calls are claimable by the next caller. As a result, callers can reasonably assume the executor liquidation path is self-funding for **LLTV=WAD** vault collateral.

Separately, a liquidator might try to avoid the executor rounding issue by bypassing the executor and setting itself as the liquidation **receiver**, so that it receives the seized vault shares directly and redeems them on its own. That direct-share fallback is not accessible in Tenor's documented VaultV2 collateral setup. The deployment uses a **receiveSharesGate** to ensure only approved take-flow contracts can receive vault shares, so a normal liquidator cannot receive seized shares directly unless it is allowlisted.

Recommended mitigation

Document the executor's pre-funding behavior as required for **LLTV=WAD** vault-share liquidations that use *MidnightVaultExecutor* as the liquidation receiver/callback. Optionally implement a path that pulls the loan token shortage up to a configured maximum from the liquidator rather than relying on atomic pre-funding.

Team response

Acknowledged and [documented](#). Since the audited commit, the balance-funded repay path was removed (leftover-funds fix from a prior review): *onLiquidate()* funds strictly from the *redeem()* return value, so atomic prefunding no longer applies. The loss-exceeds-bonus edge (including the **LLTV=WAD** rounding case) now fails closed with **RepayExceedsRedeemed**, as documented.

TRST-L-4 *TenorRouter* lacks continuous-fee slippage protection

- **Category:** Race conditions
- **Source:** *TenorRouter.sol*
- **Status:** Fixed

Description

Midnight charges a continuous fee when a take creates new credit for the buyer. The fee is taken from the market state at execution time and is added to the buyer's pending fee balance.

```
uint256 buyerCreditIncrease = UtilsLib.zeroFloorSub(units, buyerPos.debt);
uint128 buyerPendingFeeIncrease =
    UtilsLib.toUint128(buyerCreditIncrease.mulDivDown(_marketState.continuousFee *
timeToMaturity, WAD));

buyerPos.pendingFee += buyerPendingFeeIncrease;
buyerPos.credit += UtilsLib.toUint128(buyerCreditIncrease);
```

This affects both buyer-side execution patterns exposed through *TenorRouter*. In a direct **MIDNIGHT_TAKE** over a sell offer, the taker is the buyer. In a first-party **TAKE_ON_BEHALF** over a buy offer where **offer.maker == initiator**, the maker is the buyer. In both cases, a higher market **continuousFee** makes the buyer's effective received value lower even though the matched **units** and asset transfers can remain within the configured router limits.

The router exposes fill limits, price bands, and **reduceOnly** protection, but it has no parameter that commits the route to an expected maximum **continuousFee** or maximum **buyerPendingFeeIncrease**. Its final price check only compares the batch's asset totals against **units**.

```
uint256 units = totals[RouterLib.FILL_UNITS];
uint256 assets = totals[sideAssetsIndex];
if (units > 0) {
    uint256 priceCeil = UtilsLib.mulDivUp(assets, WAD, units);
    if (priceCeil > params.maxPrice) {
        revert PriceSlippageExceeded(priceCeil, params.minPrice, params.maxPrice);
    }
    uint256 priceFloor = UtilsLib.mulDivDown(assets, WAD, units);
    if (priceFloor < params.minPrice) {
        revert PriceSlippageExceeded(priceFloor, params.minPrice, params.maxPrice);
    }
}
```

By contrast, *BaseMigrationRatifier* explicitly accounts for the current continuous fee when it validates rates for Midnight-target lend flows.

```
uint256 continuousFee = MORPHO_MIDNIGHT.continuousFee(marketId);
if (continuousFee == 0) return WAD;
uint256 timeToMaturity = UtilsLib.zeroFloorSub(offer.market.maturity,
block.timestamp);
uint256 fee = continuousFee * timeToMaturity;
if (fee >= WAD) revert InvalidTargetMaturity();
return WAD - fee;
```

That ratifier-side accounting does not give *TenorRouter* users a generic inclusion-time bound. If the market's **continuousFee** is raised between offchain quoting and onchain execution, a buyer can receive the intended **units** while accepting a larger pending fee than expected.

The impact is bounded by Midnight governance and the protocol's maximum continuous fee, so this is a low-severity execution-quality issue. But it can still cause user-facing routes to execute at a worse effective rate than the buyer intended.

Recommended mitigation

Add explicit continuous-fee protection to *TenorRouter* execution. For example, include a maximum acceptable **continuousFee** in each buyer-side action, and check it against the touched market before dispatching the take. Performing the check before every take is important since theoretically the fee could be changed in between takes. The **TAKE_ON_BEHALF** route will be deprecated and does not require a mitigation.

Team response

Fixed in [PR12](#) and [documented](#) that **maxContinuousFee** is only checked once.

Mitigation review

The recommended continuous fee protection has been implemented. It is only checked before the first take is dispatched, which means a Midnight governance action executed as part of a callback could change the continuous fee mid-execution, making later takes observe a different continuous fee. This has been acknowledged and documented.

TRST-L-5 Authorized delegate settings persist after Midnight authorizations are revoked

- **Category:** Access control issues
- **Source:** IntentSettler.sol, MigrationIntentRatifier.sol, MarketMakingPolicy.sol
- **Status:** Acknowledged

Description

Several Tenor contracts let either the user or an address currently authorized by the user on Midnight write persistent user settings. The authorization check is performed only at write time. The stored setting does not record who set it and is not invalidated if that delegate later loses Midnight authorization.

In *IntentSettler*, an authorized delegate can approve a ratifier for the user. The approval is stored in **isAuthorized[user][ratifier]** independently from Midnight authorization.

```
function setIsAuthorized(address authorized, bool newIsAuthorized, address onBehalf)
external {
    if (msg.sender != onBehalf && !MORPHO_MIDNIGHT.isAuthorized(onBehalf,
msg.sender)) revert Unauthorized();
    isAuthorized[onBehalf][authorized] = newIsAuthorized;
    emit SetIsAuthorized(msg.sender, onBehalf, authorized, newIsAuthorized);
}
```

Later settlement only checks that the ratifier remains approved by *IntentSettler*. It does not check whether the address that created the approval is still trusted by the user.

```
if (!isAuthorized[intent.user][intent.ratifier]) revert RatifierNotAuthorized();

IIntentRatifier(intent.ratifier)
    .onIntentRatify(onBehalf, takerCallback, intent, offerData.offer,
takerCallbackData, ratifierData);
```

The same pattern exists for migration parameters. *MigrationIntentRatifier.setParams()* accepts writes from a currently authorized delegate and stores standing migration consent under **userParams**.

In *MarketMakingPolicy*, a currently authorized delegate can set a per-market curve for the user, and that curve remains the source of rates until cleared or overwritten.

This creates a lingering backdoor risk. While it is authorized, a delegate can install an attacker-controlled or overly permissive ratifier, permissive migration parameters, or an unfavorable market-making curve. Subsequent revocation of the malicious delegate does not clear those settings.

Recommended mitigation

At a minimum, document that revoking Midnight authorization for a delegate does not revoke or invalidate settings that delegate previously installed in *IntentSettler*, *MigrationIntentRatifier*, or *MarketMakingPolicy*. And user interfaces should surface these lingering settings and guide users to review ratifier approvals, migration parameters, and market-making curves when removing a delegate.

A stronger onchain mitigation is to track the setter for each delegated entry and require that setter to remain trusted when the entry is used. For example, *IntentSettler* could store the address that created each ratifier approval and treat the approval as inactive if that address is neither the user nor still authorized by the user on Midnight. The same pattern can be applied per **userParams** entry and per curve.

If this stronger approach is used, it must be documented that authorization revocation on Midnight can have unexpected consequences. For example, *BaseMigrationRatifier* is intended to be used with multiple routes configured at once. If the routes have been configured by different authorized users, deauthorizing just a subset of the users can leave *BaseMigrationRatifier* in an unsafe state. On balance, the stronger revocation semantics can provide improved security for users in emergency situations if the flipside of selectively revoked settings is acceptable.

Team response

Acknowledged. Settings installed by a delegate intentionally persist after Midnight revocation; users and interfaces are expected to review standing settings when removing a delegate.

TRST-L-6 Dust donations can grief assets==type(uint256).max repayments

- **Category:** Griefing issues
- **Source:** MidnightAdapterBase.sol
- **Status:** Acknowledged

Description

MidnightAdapterBase.midnightRepay() supports two max-style repayment sentinels. If **debt == type(uint256).max**, it resolves the repayment amount from the initiator's actual Midnight debt. If **assets == type(uint256).max**, it instead resolves the repayment amount from the adapter's full loan-token balance.

```
if (debt == type(uint256).max) {
    units = MORPHO_MIDNIGHT.debtOf(IdLib.toId(market, INITIAL_CHAIN_ID,
address(MORPHO_MIDNIGHT)), onBehalf);
} else if (debt != 0) {
    units = debt;
} else if (assets == type(uint256).max) {
    units = IERC20(market.loanToken).balanceOf(address(this));
} else if (assets != 0) {
    units = assets;
}
```

The **assets == type(uint256).max** branch is sensitive to donations since anyone can transfer ERC-20 tokens directly to the adapter. After the adapter resolves **units**, *Midnight.repay()* subtracts that amount from the user's debt before pulling tokens from the payer.

```
position[id][onBehalf].debt -= UtilsLib.toUint128(units);
marketState[id].withdrawable += UtilsLib.toUint128(units);

address payer = callback != address(0) ? callback : msg.sender;
...
SafeTransferLib.safeTransferFrom(market.loanToken, payer, address(this), units);
```

If **units** is even one wei greater than **position[id][onBehalf].debt**, the debt subtraction underflows and the repay reverts. A victim bundle that expects the adapter balance to fit the user's debt can therefore be grieved by a dust transfer to the adapter before the bundle executes. Given that the **debt == type(uint256).max** flow already clamps to the user's debt, it is reasonable for users to expect the same behavior from the **assets == type(uint256).max** flow.

Recommended mitigation

Cap the balance-based repay sentinel by the user's outstanding Midnight debt before calling *Midnight.repay()*, and document that the repayment should be bundled with a sweep of excess tokens.

Team response

Acknowledged. Guaranteed full close-outs should use the units sentinel (**debt** = **type(uint256).max**), which resolves from actual debt and is donation-proof. **assets** = **type(uint256).max** follows the implicit adapter-balance sweep convention shared by first-party bundler adapters (cf. Morpho **GeneralAdapter1**) and is intentionally unchanged.

Additional recommendations

TRST-R-1 Use a consistent token approval pattern

The codebase uses several token approval patterns for protocol and vault integrations. Some paths approve exact amounts, some approve **type(uint256).max**, some reset approvals to zero after use (see [TRST-M-5](#)), and some first check whether the current allowance is already max before setting a max approval.

For example, *MidnightAdapterBase.midnightSupplyCollateral()* approves Midnight for **type(uint256).max** even though the resolved **assets** amount is known.

```
SafeERC20.forceApprove(IERC20(collateralToken), address(MORPHO_MIDNIGHT),
type(uint256).max);
MORPHO_MIDNIGHT.supplyCollateral(market, collateralIndex, assets, onBehalf);
```

In *TenorRouter*, the direct payer path temporarily grants a max approval and then resets it to zero after the take returns or reverts. This zero-reset pattern is not used consistently elsewhere and introduces TRST-M-5.

```
if (routerIsPayer) SafeERC20.forceApprove(IERC20(loanToken),
address(_MORPHO_MIDNIGHT), 0);
```

Other paths use exact approvals, such as the *MidnightVaultExecutor* deposit path.

```
asset.forceApprove(vault, assets);
depositedShares = vaultContract.deposit(assets, address(this));
```

A fourth pattern appears in the flash-loan adapter path, where the code checks whether the approval is already **type(uint256).max** before setting it to max.

```
if (IERC20(tokens[i]).allowance(address(this), address(MORPHO_MIDNIGHT)) !=
type(uint256).max) {
    SafeERC20.forceApprove(IERC20(tokens[i]), address(MORPHO_MIDNIGHT),
type(uint256).max);
}
```

It is recommended to apply a consistent approval pattern. Approvals from the router to Midnight should always unconditionally be **type(uint256).max**. And other approvals for vaults and Morpho Blue or for Midnight outside of the router should be limited to the amount that is subsequently pulled. Within the context of a given **initiator**, unauthorized users can't pull funds from the router to Midnight, making the max approval safe and avoiding the issue in [TRST-M-5](#).

TRST-R-2 Zero buyer-asset takes should be supported

Midnight.take() can execute with **units > 0** and **buyerAssets == 0**. In that case, the buyer receives the units at zero asset cost. Several Tenor buyer callbacks currently reject that fill shape before reaching their route-specific logic.

```
if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
if (buyerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();
```

The affected callbacks are *BorrowMidnightToBlueCallback*, *LendMidnightRenewalCallback*, *LendVaultToMidnightCallback*, and *MidnightWithdrawVaultSharesCallback*.

Since **units > 0** and **buyerAssets == 0** is allowed in Midnight and beneficial for the buyer, the callbacks should support this flow, even though it is not sensible for a seller to act as the counterparty.

If the change is implemented, it must be ensured that the whole flow is compatible with **buyerAssets == 0**. E.g., *BorrowMidnightToBlueCallback* would attempt to borrow zero assets from Blue and revert.

TRST-R-3 Verify the *previewMint()* and *mint()* asset equality assumption

MidnightVaultExecutor documents a vault safety requirement for the mint path: *previewMint(shares)* must equal the assets consumed by *mint(shares)*, so no dust is left behind.

```
/// - `deposit`, `mint` and `redeem` move exactly the assets/shares they report. The
/// executor pulls
/// `previewMint(shares)` on the mint path, which equals what `mint` consumes in the
/// same transaction, so no dust is
/// left behind.
```

The mint path relies on that assumption by previewing the required assets, pulling exactly that amount from **msg.sender**, approving the vault, and then calling *mint()* without checking the returned asset amount.

```
usedAssets = vaultContract.previewMint(shares);
asset.safeTransferFrom(msg.sender, address(this), usedAssets);
asset.forceApprove(vault, usedAssets);
vaultContract.mint(shares, address(this));
depositedShares = shares;
```

The documented condition of parity between *previewMint()* and *mint()* is insufficient to guarantee that they use the same assets in the flow above. The *safeTransferFrom()* which is executed in between *previewMint()* and *mint()* can theoretically affect the vault's exchange rate and make the assets consumed by *mint()* lower than the value returned by *previewMint()*. It is recommended to check that the return value of *mint()* is equal to **usedAssets** to enforce the assumption on-chain.

TRST-R-4 Code quality improvements

LinearInterpolationLib.interpolate() assigns `y = values[lastIndex]` after already handling the upper clamp. With well-formed strictly increasing **knots**, the loop always returns for the remaining input range, so the assignment is dead code and can be removed.

Constructor zero-address validation should be consistent across the codebase. *TenorRouter* [stores](#) **intentSettler** and **morphoMidnight** without explicit zero-address checks, while other contracts do validate constructor addresses.

onSell() callbacks should enforce their documented receiver assumptions instead of relying only on comments. For callbacks that must receive seller assets before continuing, name the **receiver** parameter and revert unless **receiver == address(this)**. Conversely, *MidnightSupplyCollateralCallback* documents that the receiver must not be the callback contract because loan proceeds sent there are permanently locked, so it should enforce **receiver != address(this)**.

MidnightSupplyCollateralCallback should also document that its pro-rata collateral supply model is intended for fills where the seller is increasing debt. If the seller is reducing existing credit or crossing from credit into debt, the **sellerAssets**-based pro-rata collateral amount does not represent the target debt increase. Consider adding a sanity check that **pendingFeeDecrease == 0** to fail early on the credit-decrease path, while documenting that this is only a partial check when the continuous fee rate is zero.

TenorRouter._execute() should fail early when **minFill > maxFill**. Currently, the router can proceed into dispatch even though the fill bounds are impossible to satisfy, then fail later through the existing fill checks. An explicit upfront validation, such as **InvalidFillBounds**, would make malformed execution parameters cheaper and easier to diagnose.

IntentSettler.take() accepts a **receiver** parameter but then checks that it satisfies a specific requirement depending on **offerData.offer.buy**. It is recommended to remove the **receiver** from the interface and to calculate it directly as **receiver = (offerData.offer.buy ? takerCallback : address(0))**.

TRST-R-5 Documentation improvements

LinearInterpolationLib.interpolate() currently [says](#) unsorted input returns garbage without reverting. Unsorted input can also revert for some values because segment arithmetic can overflow. The comment should say malformed curves may return meaningless values or revert.

MAX_FEE_RATE_MIDNIGHT_TO_BLUE is [documented](#) as the maximum fee for Midnight-to-Blue actions, but *BaseMigrationRatifier._maxFeeRate()* also applies it to **LEND_MIDNIGHT_TO_VAULT_CALLBACK**. The constant-level comment should describe the shared class accurately and the constant may be renamed to encompass the broader meaning.

Currently, there is no documentation for supported / unsupported token behaviors. It is recommended to document this prominently.

[MidnightVaultExecutor](#) and [MidnightSupplyVaultSharesCallback](#) document share-price manipulation resistance, but [MidnightWithdrawVaultSharesCallback](#) should state the same assumption because it also settles at the vault rate available at execution time.

The [MidnightSupplyVaultSharesCallback documentation](#) states a requirement for the vault, saying it is not guaranteed by ERC-4626 compliance. This is incorrect, ERC-4626 compliance requires a correct *deposit()* return value.

Callback documentation should say when a callback is valid for taker-side use as well as maker-side use. Current [documentation](#) implies maker-side use only. This recommendation applies to all callbacks.

[MidnightSupplyVaultSharesCallback](#) should document that it intentionally has no **maxLtv** or account-health boundary. Its intended use is narrow: vault shares are used as collateral to borrow the underlying loan token of the vault. If the position has other collateral, a different collateral mix, or price changes between configuration and fill, the same **additionalDepositPercent** can result in different account health.

[BorrowMidnightToBlueCallback](#) should document its multi-collateral behavior explicitly. The source Midnight market may list and hold multiple collaterals, but the callback migrates only the collateral matching the target Blue market collateral token. As a result, the resulting Blue position can have materially worse LTV than the source position if other source collateral remains behind.

[BaseMigrationRatifier._extractCallbackContext\(\)](#) decodes **LEND_MIDNIGHT_RENEWAL_CALLBACK** callback data using **IBorrowMidnightRenewalCallback.CallbackData**. This is currently correct because the borrow and lend renewal callback data structs are identical. That cross-interface dependency should be documented so future changes to either struct do not silently break ratification.

[StaticRatePolicy](#) and [MarketMakingPolicy](#) should document their intended route shapes and use cases. In particular, [MarketMakingPolicy](#) should be described as intended for [LendMidnightToVaultCallback](#) and [LendVaultToMidnightCallback](#). [StaticRatePolicy](#) should document the common three-migration setup where Midnight-to-Midnight, Midnight-to-Blue, and Blue-to-Midnight routes are configured.

[TakeMathLib](#) says [sizers never revert](#) and saturate on overflow. That should be qualified to the intended valid-offer and **uint128**-scale budget domain, because extreme inputs such as **type(uint256).max** budgets can still overflow in some math paths.

[TenorRouter](#) should document the exact scope of **Action.allowRevert**. Reverts inside the inner [IntentSettler.take\(\)](#) or [Midnight.take\(\)](#) are caught, but local sizing and validation before the try/catch can still abort the whole batch. For example, an out-of-range tick can revert during local price math, while dynamic Midnight execution failures such as seller price underflow after settlement fees are handled as action reverts.

IRenewalCadence.nearestBoundary() is [specified](#) as returning the largest cadence point at or before the timestamp. *BaseMigrationRatifier._ratifyWindow()* still checks **renewalPeriodStart > block.timestamp** after calling it. The comment should explain that this is an invariant check that should never fail for a compliant cadence implementation, not an expected runtime branch.

Centralization risks

TRST-CR-1 *MigrationIntentRatifier* owner

MigrationIntentRatifier inherits *BaseMigrationRatifier*, whose owner can configure protocol migration fees through *setFeeConfig()*. Fee config is keyed by **callback** and **tenorMarketId**, with a **bytes32(0)** entry acting as the default when no market-specific recipient is set.

```
function setFeeConfig(address callback, bytes32 tenorMarketId, uint256 _feeRate,
address _feeRecipient)
    external
    onlyOwner
{
    if (_feeRate > _maxFeeRate(callback)) revert InvalidFeeConfig();
    if (_feeRate > 0 && _feeRecipient == address(0)) revert InvalidFeeConfig();
    FeeConfig storage slot = feeConfigs[callback][tenorMarketId];
    slot.feeRecipient = _feeRecipient;
    slot.feeRate = uint96(_feeRate);
    emit FeeConfigSet(callback, tenorMarketId, _feeRate, _feeRecipient);
}
```

The maximum configurable fee depends on the migration direction. Midnight exit flows are capped at zero, while other migration and renewal flows are capped at 50% of the interest component.

```
uint256 constant MAX_FEE_RATE = 0.5e18;
uint256 constant MAX_FEE_RATE_MIDNIGHT_TO_BLUE = 0;
```

The owner can therefore change fee recipients and set fees up to the hard-coded caps. The ratifier performs the user rate check after applying fees, so this privilege cannot weaken the user-supplied rate bounds.

TRST-CR-2 Pausable interest rate policy owner

The owner of a pausable interest rate policy controls the pauser set and is the only role that can unpause the policy. The owner can add or remove pausers through *setPauser()* and can restore rate queries through *unpause()*.

```
function setPauser(address pauser, bool allowed) external onlyOwner {
    isPauser[pauser] = allowed;
    emit PauserSet(pauser, allowed);
}

function unpause() external onlyOwner {
    if (!paused) revert NotPaused();
    paused = false;
    emit Unpaused(msg.sender);
}
```

The owner can therefore decide which risk managers can pause renewals and when a paused renewal policy resumes. This centralizes availability control and migration risk management for users who choose a pausable policy.

TRST-CR-3 Pausable interest rate policy pauser

Any configured pauser of a pausable interest rate policy can call *pause()*, which causes *getRate()* to revert.

```
function pause() external {  
    if (!isPauser[msg.sender]) revert OnlyPauser();  
    if (paused) revert AlreadyPaused();  
    paused = true;  
    emit Paused(msg.sender);  
}
```

If a user opts into using a pausable rate policy in their migration configuration, they trust the pauser set to detect risks like pending bad debt or collateral de-pegs in the target market. Failing to pause when such risks are present can cause financial losses for affected users. Similarly, pausing when there is no risk can cause losses, e.g., by preventing renewal of a position which then gets liquidated after maturity.

Systemic risks

TRST-SR-1 External token risks and assumptions

The protocol relies on external ERC20 tokens as collateral and loan tokens. These tokens are outside the protocol's control and must continue to satisfy their functional and economic assumptions. This includes behaviors such as not charging transfer fees and stablecoins maintaining their peg. The protocol is compatible with all tokens that are supported by Midnight.

TRST-SR-2 External protocol risks

Tenor composes with external protocols. The integrated protocols are Morpho Midnight, Morpho Blue, and Morpho Vaults.

As a result, Tenor users and integrators inherit the risks of these external protocols. Since it is a periphery wrapper around those protocols, Tenor is directly affected by a third-party security vulnerability or economic issue even if the Tenor contracts themselves are safe.

TRST-SR-3 Borrower migration callbacks don't check account health

The Tenor migration and renewal paths intentionally do not include a user-configurable health check. Instead, the callbacks rely on the operational assumption that source and target markets are economically equivalent or near-equivalent in their risk parameters such that a pro-rata migration of collateral does not lead to unexpected account health in the source or target market. Only the standalone *MidnightSupplyCollateralCallback* supports a health check.

It must be understood that borrower health, even if source and target market risk parameters are equivalent, generally drifts down due to an increased interest burden in the target market and fees. It is the user's responsibility when setting up migrations to understand the consequences on the account health. In the worst case, unsafe configuration can lead to liquidations and corresponding losses.